ZKVeil: A Privacy-Preserving Compliance Verification Scheme for Blockchain-Enabled Supply Chain Transactions

Dongyu Cao, *Graduate Student Member, IEEE*, Bixin Li, *Member, IEEE*, Huijie Zhang, Yong Wang, and Lulu Wang

***Abstract*—Blockchain technology improves supply chain management by ensuring the immutability of transaction records and facilitating process tracking. However, the transparency of blockchain raises significant privacy concerns, as sensitive information such as buyer and supplier qualifications, product specifications, and transaction amounts is often exposed. Compliance verification, which needs access to specific sensitive data for compliance checks, becomes challenging in blockchain-based privacy-preserving supply chains. This paper introduces ZKVeil, an innovative scheme utilizing zero-knowledge proof technology to maintain the confidentiality of sensitive information while ensuring compliance verification. Additionally, ZKVeil uses decentralized identifiers and verifiable credentials to ensure the authenticity of transaction data. A theoretical security analysis demonstrates the effectiveness of ZKVeil in safeguarding real sensitive data and ensuring compliance with regulations. To evaluate the performance of our scheme, we implement ZKVeil on a private blockchain of 100 nodes. Taking the shipbuilding supply chain transaction as an example, the experimental results demonstrate that ZKVeil incurs low gas consumption, execution time, and memory overhead.**

***Index Terms*—Blockchain, supply chain, privacy protection, compliance verification.**

I. Introduction

SUPPLY chains are complex networks that involve multiple stakeholders, including manufacturers, suppliers, distributors, and customers. Supply chains have become a crucial component of the modern economy with the rapid advancement of globalization and digitalization. In cross-domain and cross-regional collaborations between participants within supply chains, it is paramount to ensure the efficient operation of information flows, capital flows, and logistics at every stage of production, circulation, and delivery [1], [2]. However, the traditional supply chain (TSC) is based on centralized management, with various stages coordinated through conventional methods and systems. As the scale expands and processes become more complex, this model reveals several issues, such as information asymmetry, data silos, lack of transparency, high intermediary costs, and significant risks associated with single points of failure. These problems not only reduce the efficiency of the supply chain, but also weaken the resilience and competitiveness of the supply chain in the face of market volatility and uncertainty [3], [4], [5].

The emergence of blockchain technology has transformed the operational model of traditional supply chain management [6], [7]. The transparency, tamper-resistance, and decentralization features of blockchain enable participants in the supply chain to share and access data in real-time, thereby eliminating data silos and ensuring consistency in information updates [8]. Specifically, the transparency of blockchain enhances the collaborative efficiency between different stages and increases the visibility of the supply chain. Its immutability strengthens the trustworthiness and integrity of supply chain data, greatly enhancing product traceability and anti-counterfeiting capabilities. The decentralization and distributed storage characteristics improve risk resistance by preventing operational disruptions caused by single points of failure, thereby boosting the supply chain's resilience in the face of uncertain events. Additionally, blockchain's smart contract technology enables the automation of transactions, reducing reliance on intermediaries and lowering costs. As a result, blockchain-enabled supply chains (BSC) have become a widely adopted and feasible supply chain management model.

Regulation in the supply chain refers to the supervision and management of processes, participants, and their behaviors in the supply chain to ensure that they comply with relevant laws and industry standards. In 2008, China's Sanlu Group's infant formula was found to contain melamine, a toxic chemical that can artificially increase the protein content of milk powder. Thousands of infants experienced health problems from consuming the contaminated formula. Subsequent investigations revealed that Sanlu's regulatory system was severely lacking, as the company neglected product quality and safety in order to reduce costs. This incident highlighted the importance of regulatory oversight in the supply chain, as it can prevent fraud and ensure product quality and safety. In BSC, transactions and business data are recorded and open to blockchain nodes, which makes it easier for regulators to enforce compliance with regulations and standards in the supply chain [9]. This transparency, while beneficial for regulatory oversight, raises

Received 20 May 2025; revised 2 January 2026 and 18 January 2026; accepted 20 January 2026. Date of publication 3 February 2026; date of current version 6 February 2026. This work was supported in part by the Key Research and Development Program of Jiangsu Province under Grant BE2021002-3 and in part by the Big Data Computing Center of Southeast University. The associate editor coordinating the review of this article and approving it for publication was Dr. Meng Li. *(Corresponding author: Bixin Li.)*

The authors are with the School of Computer Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: dongyucao@seu.edu.cn; bx.li@seu.edu.cn; huijiezhang@seu.edu.cn; wangyong99@seu.edu.cn; wanglulu@seu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3660595

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

significant privacy concerns. Sensitive information such as product specifications, transaction amounts, and personal data of buyers and suppliers are often exposed on the blockchain. This can lead to potential misuse of data and identity theft.

To effectively protect the private data of the supply chain from being leaked or tampered with, many privacy protection methods [10], [11], [12] have been proposed to record encrypted sensitive information on the blockchain. These approaches typically rely on cryptographic techniques such as attribute-based encryption and digital signatures to ensure that sensitive information remains confidential. However, when regulatory compliance is required, existing solutions exhibit fundamental limitations. **On the one hand**, some schemes [12], [13], [14] focus solely on data confidentiality and upload ciphertexts directly to the blockchain without considering regulatory requirements. As a result, regulators must decrypt and inspect on-chain data during supervision, which not only introduces heavy computational overhead but also incurs significant delays and operational costs, severely reducing regulatory efficiency. **On the other hand**, other approaches [11] primarily focus on accountability by enabling a trusted authority to disclose the identity of a signer when misbehavior is detected. However, such identity-disclosure-based mechanisms do not provide regulatory capabilities to assess compliance with predefined rules, such as whether products satisfy required production specifications or conform to regulatory policies.

Zero-knowledge proofs (ZKPs) are cryptographic protocols that enable a prover to convince a verifier of the correctness of a statement without revealing any information beyond the validity of the statement itself. ZKPs allow participants to prove satisfaction of regulatory predicates while revealing nothing beyond the verification outcome, making them particularly suitable for decentralized regulatory enforcement. However, ZKPs alone do not provide guarantees on the authenticity of the underlying data. Specifically, a proof only establishes that the prover possesses a witness satisfying the specified relation, without ensuring that the witness corresponds to truthful or authoritative real-world information. In contrast, regulatory compliance requires verifiable authenticity of identities, qualifications, and product certifications. Decentralized identifiers (DIDs) and Verifiable credentials (VCs) provide strong data authenticity guarantees, as credential claims are digitally signed by trusted issuers such as certification authorities. Directly disclosing VCs for verification reveals sensitive identity or business information, which conflicts with privacy requirements in supply chain scenarios. Embedding VC authenticity verification into ZKPs can verify policy satisfaction without accessing underlying credential data or relying on trusted compliance checkers.

Building on these considerations, we propose ZKVeil, a privacy-preserving compliance verification scheme for blockchain-enabled supply chain transactions combining ZKPs and VCs. Specifically, credential authenticity is ensured by issuer signatures, while ZKPs are used to verify compliance over credential attributes without disclosing their values. This combination enables authenticated yet privacy-preserving compliance verification, achieving both data trustworthiness and minimal disclosure. The key contributions of ZKVeil are summarized as follows:

- We propose a privacy-preserving compliance verification scheme for BSC, called ZKVeil. ZKVeil leverages ZKPs to ensure that sensitive information, including identity, product specifications, and transaction amounts, remains confidential while still verifying compliance with regulatory requirements.
- We design a system model that includes buyers, suppliers, blockchain, and two authentication authorities. This model leverages DIDs and VCs to ensure the authenticity of sensitive information.
- We conduct a thorough security analysis of ZKVeil, demonstrating its ability to protect sensitive information and ensure compliance with regulations. We also evaluate the performance of ZKVeil, showing that it can be effectively implemented in supply chain transactions.

Section II discusses related work. Section III provides the necessary background knowledge. Section IV presents the model and goals of ZKVeil. Section V describes the construction of ZKVeil. Section VI analyzes the security of ZKVeil. Section VII presents the performance evaluation results. Section VIII discusses the security goals of ZKVeil. Finally, Section IX concludes the paper.

II. Related Work

In BSC, the open nature of blockchain raises concerns about privacy protection. Existing studies have proposed various solutions to address these issues.

In supply chains, multi-hop information transmission allows nodes to share and transfer information through multiple steps. However, due to the lack of trust between stakeholders, this often results in incomplete or inaccessible product tracking information. To address this, Bader et al. [10] proposed a privacy-preserving architecture called PrivAccIChain. It combines smart contracts and attribute-based encryption, offering an adaptable configuration that allows adjustment between transparency and data privacy. The scheme also sets up a detached judge to resolve disputes manually regarding access rights management.

The transparency of blockchain itself raises concerns about the alteration or leakage of product information and participant identity information. The ProChain scheme [14] utilizes zero-knowledge proof technology, allowing participants to enter the system using pseudonyms, thereby protecting their identity information. At the same time, attribute-based encryption technology is used to encrypt product information, ensuring that the product data on the blockchain is effectively protected.

Although some existing blockchain systems based on attribute-based encryption can solve some privacy issues, they either introduce additional security problems [15] or lack feasibility analysis [16]. Xu et al. [13] proposed a blockchain system based on Multi-Authority Attribute-Based Encryption [16], aimed at protecting data privacy in Internet of things (IoT) supply chains. The scheme designs a four-way trade-off optimization framework to ensure that while enhancing privacy protection, key factors such as decentralization, scalability, and storage consumption are not significantly affected.

Existing blockchain-based anti-counterfeiting supply chain methods have not considered privacy issues [17], and those with privacy protection have not addressed anti-counterfeiting

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING TYPICAL SCHEMES

Paper	Field	Identity Privacy	Product Privacy	Transaction Privacy	Data Authenticity	Regulation
PrivAccIChain [10]	Electric Vehicle	✓	✓	✗	✗	Manual dispute resolution by a judge
APPB [11]	Vehicle	✓	✓	✗	✗	Accountability via identity disclosure
TPPSUPPLY [12]	General	✓	✗	✓	✗	✗
Xu et al. [13]	Internet of Things	✗	✓	✗	✗	✗
ProChain [14]	General	✓	✓	✗	✗	✗
ZKVeil(Ours)	General	✓	✓	✓	✓	Compliance verification over ciphertext

problems [18], [19], [20]. APPB [11] builds blockchain-based supply chains that can both prevent counterfeiting and protect privacy. For privacy protection, APPB uses symmetric-key encryption and group signatures [21] to protect the privacy of product sales information and the business relationships between manufacturers, agents, and retailers. For anti-counterfeiting, the ID of each product should be linked to unique ownership. Buyers can obtain product information and track the product flow through its product ID. APPB also sets up a trusted supervisor, who serves as the manager of the group signature and can disclose the identity of the signer. Therefore, when malicious sellers emerge, the trusted institution can obtain the malicious party's identity information. Similarly, Mohit et al. [22] proposed a new mechanism that comprehensively considers traceability and ownership to ensure the privacy of information shared between supply chain members. The product owner can trace the product and transfer ownership of it, which limits the entry of counterfeit products into the supply chain to some extent.

Many existing supply chain frameworks have issues such as poor traceability, lack of real-time information, and insufficient privacy protection [23], [24]. Sezer et al. [12] proposed a framework, TPPSUPPLY, designed to provide a solution that ensures traceability of supply chain information and protects privacy. This framework uses smart contracts and digital signatures to implement digital signing and verification both on-chain and off-chain, thereby ensuring the privacy and security of information. Additionally, the framework allows the adjustment of anonymity or traceability according to user requests. Li et al. [25] proposed an efficient and secure port supply chain data management scheme. This scheme utilizes dynamic searchable encryption for port data sharing. In addition, access control rules are defined and automatically executed in the smart contract on the blockchain. These policies allow decentralized permission management and efficient access control of on-chain operations, enabling fine-grained access control and secure data sharing.

With the increase in the growing global demand for vaccines, vaccine supply chain management faces complex challenges, especially in terms of data sharing and privacy protection. Yong et al. [26] designed a new vaccine supply chain intelligent supervision system, and designed an Ethereum-based personal vaccination record query and vaccine circulation smart contract for consumers, vaccine agencies, and governments to trace vaccine operation records. To protect privacy, the personal information of the vaccinators is not recorded on the blockchain. Wen et al. [27] designed a blockchain-based vaccine supply chain architecture that leverages the decentralized nature and immutability of blockchain to ensure the security of vaccine information. This architecture uses encryption algorithms to protect the vaccination information and insurance data of recipients and sets up regulatory agencies as participants. A regulatory contract is also designed, primarily to record the circulation process of the vaccine, focusing on two functions: one is to input the product transportation information, and the other is to query the transportation status of the product. If adverse reactions occur after vaccination, the regulatory agency conducts a review. The review process involves a thorough examination of the entire lifecycle of the vaccine to identify any potential illegal operations.

Some existing typical schemes are listed in Table I, which illustrates that most of the existing studies overlook regulatory requirements and data authenticity. Among the studies that consider regulation, supply chain regulation focuses on ensuring oversight across the entire lifecycle of the supply chain. However, different regulatory approaches are applied at each stage of the supply chain. Additionally, all privacy data are visible to designated regulatory nodes. This open-access model lacks mechanisms to limit the authority of regulatory nodes, often leading to the unintended overexposure of sensitive data. In contrast, ZKVeil provides comprehensive privacy protection while supporting regulation through compliance verification over ciphertext, thereby avoiding plaintext disclosure and preserving data confidentiality.

III. PRELIMINARIES

This section provides essential background knowledge to facilitate a better understanding of ZKVeil.

A. Blockchain

Blockchain is a distributed ledger technology that enables secure and transparent transactions without the need for intermediaries [28]. It consists of a chain of blocks, where each block contains a list of transactions and a cryptographic hash of the previous block. This structure ensures the integrity and immutability of the data stored on the blockchain. Blockchain technology has been widely adopted in various industries, including finance, supply chain management, healthcare, etc. Blockchain technology is characterized by decentralization, transparency, and immutability. It operates on a peer-to-peer network in which each participant maintains a ledger, eliminating the need for a central authority and reducing the risk of single points of failure. All transactions are transparently recorded and visible to authorized participants, while the append-only ledger structure ensures that once a transaction is

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

confirmed, it cannot be altered or deleted, thereby providing a tamper-resistant record of system activities.

B. DIDs and VCs

Decentralized Identifiers (DIDs)[1] are a new type of identifier that enables self-sovereign identity on the blockchain. DIDs are unique identifiers that are associated with cryptographic keys and can be used to authenticate users and entities in a decentralized and privacy-preserving manner [29]. Verifiable Credentials (VCs) are digital credentials that are issued by trusted entities and can be presented by users to prove their identity, qualifications, or specifications. The various players in a decentralized identity ecosystem include:

(1) Holders: Individuals who own and use pieces of identity information. Users can keep various identifiers in a digital wallet and share them on an as-needed basis.

(2) Issuers: Organizations and institutions that issue credentials and claims to users. This can be your local Tax Office, an employer, an academic institution, or any entity that can issue identity information.

(3) Verifiers: Third parties who need identity information to establish trust and grant access to services. For example, an online shop may need proof of your age or citizenship before allowing you to purchase certain items. Any information you provide would need to be verified properly.

Holders, issuers, and verifiers interact within the context of DIDs and VCs. Holders manage their DIDs and store VCs in a digital wallet, issuers create and sign VCs to provide verified identity information, and verifiers use DIDs and VCs to authenticate and validate the information for establishing trust and granting access to services.

C. Zero-Knowledge Proofs

Zero-Knowledge Proofs (ZKPs) [30], [31] are cryptographic protocols that enable a prover to prove the validity of a statement without revealing any information about the statement itself. ZKP is used to demonstrate the knowledge of a secret without disclosing the secret itself, thereby preserving the confidentiality of the information. Zero-Knowledge Succinct Non-interactive ARguments of Knowledge (ZK-SNARKs) [32] are a type of ZKP that provide succinct proof size and fast verification time, making ZK-SNARKs suitable for privacy-preserving applications on the blockchain. A ZK-SNARK scheme contains three algorithms: Setup, Prove, and Verify.

(1) $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{C})$: The setup algorithm takes a security parameter 1^λ and a circuit C as input, and generates a common reference string (CRS) crs that is used by the prover and verifier to generate and verify proofs. CRS can generate a proving key pk and a verification key vk.

(2) $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{x}, \mathsf{w})$: The Prove algorithm takes the proving key pk, a statement x, and a witness w as input and generates a proof π that demonstrates the validity of the statement.

(3) $\{0, 1\} \leftarrow \mathsf{Verify}(\mathsf{crs}, \pi, \mathsf{x})$: The Verify algorithm takes the verification key vk, a statement x, and a proof π as input and verifies the proof without learning any information about the statement.

[1]https://www.w3.org/TR/did-1.1/

ZK-SNARKs satisfy three properties: completeness, soundness, and zero-knowledge. Completeness ensures that if the statement is true, the prover can convince the verifier with high probability. Soundness guarantees that if the statement is false, no dishonest prover can persuade the verifier with a non-negligible probability. Zero-knowledge ensures that the verifier gains no information about the witness or any other details beyond the validity of the statement.

D. BlockMaze

BlockMaze [33] is a novel privacy-preserving blockchain designed to address the privacy issues of account-model blockchains. This blockchain introduces a dual-balance model, where each account has both a plaintext balance and a zero-knowledge balance. This design supports transactions that obfuscate transaction amounts and prevent linkage between senders and recipients. BlockMaze has four polynomial-time algorithms: Mint, Send, Deposit, and Redeem. The privacy-preserving transaction process between A and B works as follows:

(1) Mint: A executes the Mint algorithm to generate a transaction, which converts A's plaintext balance into zk_balance.

(2) Send: A runs the Send algorithm to produce a transaction, and a fund commitment (cmt_v) to be recorded on the blockchain. This cmt_v represents the fund designated for B.

(3) Deposit: B selects multiple cmt_v from the blockchain and organizes them into a Merkle tree. B then generates a Merkle proof, confirming that the cmt_v is part of the tree. Using this proof, B runs the Deposit algorithm to move the fund into B's zk_balance.

(4) Redeem (Optional): If desired, B can use the Redeem algorithm to transfers B's zk_balance back to its plaintext balance.

BlockMaze directly redesigns the account-model blockchain protocol by introducing a dual-balance abstraction and an account-based amount commitment mechanism. This design natively supports both account balance privacy and sender–recipient unlinkability, while preserving explicit account states. Such properties are particularly relevant for ZKVeil, where regulatory predicates must be enforced over committed amounts and payment privacy must be combined with delivery-dependent settlement logic at the account level. Existing privacy-preserving designs typically address this challenge through either (i) UTXO-style abstractions based on independent shielded notes, such as Zcash,[2] or (ii) note-based private execution frameworks deployed at higher layers, such as Aztec.[3] While effective in their respective settings, these approaches do not maintain account balance semantics, and are therefore not directly applicable to account-model blockchains that rely on a single-account abstraction.

IV. Models and Goals

In this section, we present the system model, threat model, and design goals of ZKVeil.

A. System Model

ZKVeil contains five types of entities: buyer (B), supplier (S), Identity Authentication Authority (IAA), Prod-

[2]https://z.cash/

[3]https://aztec.network/

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

uct Certification Authority (PCA), and the blockchain network.

Supplier S is responsible for providing products to the buyer B. The supplier must ensure that the product specifications meet regulatory and industry standards. The buyer B is responsible for making payments to the supplier and ensuring that the payment amount meets regulatory requirements.

IAA and PCA are essentially represented by an account in the blockchain. They represent a trusted entity that issues DIDs and VCs to buyers and suppliers. IAA is responsible for checking the identity and qualifications of buyers and suppliers, while PCA is responsible for checking the product specifications and quality.

The blockchain network is a decentralized and distributed ledger that records all transactions between buyers and suppliers. It consists of multiple miners who follow a secure consensus algorithm to maintain the integrity and security of the blockchain.

B. Threat Model

In order to capture the security and privacy requirements of ZKVeil, we define the following threat model:

We assume that suppliers can be semi-honest or malicious, meaning they follow ZKVeil but may attempt to forge compliance proofs, infer private buyer data, or deny delivery after receiving payment

We assume that buyers can be arbitrarily malicious and act in their best interests. They may attempt to bypass qualification requirements, commit to invalid transaction amounts, or refuse to acknowledge delivery

IAA and PCA are assumed to be trusted. In reality, they are usually government agencies or state administrations that have the authority to issue DIDs and VCs. They are responsible for verifying the identity and qualifications of buyers and suppliers and the quality of products.

Multiple blockchain miners follow a secure consensus algorithm to maintain the blockchain. Adversaries cannot compromise the majority of miners to bring down the overall blockchain system.

C. Design Goals

Based on the above threat model, ZKVeil intends to achieve the following privacy and security goals:

1) (G1) Ensuring Identity Privacy and Compliance: The disclosure of sensitive identity information (such as age, income, credit rating, etc.) can lead to risks such as identity theft and financial loss. At the same time, regulators need to conduct identity checks on buyers and suppliers. For example, food suppliers are also required to have valid production licenses and health certifications. In ZKVeil, the identity information of buyers and suppliers is kept confidential while ensuring compliance with regulatory requirements. Specifically, participants generate ZKPs π_{supplier} and π_{buyer} that demonstrate their qualifications satisfy regulatory policies Σ and $\mathcal{T}_P$ respectively (formalized in Section V), without disclosing underlying credential data.

2) (G2) Ensure Product Specification Privacy and Compliance: Product Specification Privacy refers to the sensitive information of the products, such as the choice of the raw materials, which is usually the core business secret of the suppliers. However, regulatory oversight of product specifications is critical to ensure product safety, quality, and adherence to industry standards. In ZKVeil, the product specifications are protected from other entities while ensuring compliance with industry standards. This is achieved through a zero-knowledge proof π_{product} that certifies specific product attributes satisfy regulatory constraints Φ (detailed in Section V), without revealing the actual specification values.

3) (G3) Ensure Transaction Amount Privacy and Compliance: Transaction amount usually involves corporate financial secrets, so direct disclosure of transaction amount should be avoided. However, the financial flow of transactions to prevent illegal activities such as money laundering and tax evasion needs to be properly monitored. In ZKVeil, the transaction amount is hidden externally, but it can still be verified through π_{amt}, proving that encrypted amounts fall within regulatory thresholds $0 \le v \le \Theta$ (detailed in Section V).

4) (G4) Data Authenticity: The identity and the product specification information should be authentic and reliable. The data should be unforgeable and verifiable by authorized entities to ensure the integrity and trustworthiness of the information. In ZKVeil, this is ensured through digital signatures embedded in Verifiable Credentials, which are issued by trusted authorities (IAA and PCA) and verified on-chain by smart contracts.

TABLE II
SYMBOLS USED IN ZKVEIL

Symbol	Description
$\mathcal{P}$	Participants in the supply chain
$\mathcal{A}$	Blockchain accounts of participants
DIDs	Decentralized Identifiers
VC	Verifiable Credentials
$Info_S$	Identity and qualification information of the supplier S
$Info_B$	Identity and qualification information of the buyer B
π_{supplier}	Zero-knowledge proof from supplier S
π_{buyer}	Zero-knowledge proof from buyer B
π_{amt}	Zero-knowledge proof of transaction amount
π_{product}	Zero-knowledge proof of product specification
Σ	Regulatory policy for transaction verification
$\mathcal{T}_P$	Regulatory policy for product specification verification
Φ	Regulatory policy for identity verification
Θ	Regulatory threshold for transaction amount verification
pk, vk	Proving key and verification key

V. ZKVEIL

In this section, we first provide an overview of ZKVeil, then explain the scheme in detail.

A. Overview of ZKVeil

In supply chains, there exist numerous scenarios that require regulation due to the complexity and sensitivity of the processes. The ZKVeil scheme focuses specifically on the supply chain transaction, which primarily involves two parties: the buyer and the supplier. The supplier provides products to the buyer, while the buyer makes a payment to the supplier. Fig. 1 presents the sequence diagram of the ZKVeil scheme, which consists of five phases: Preparation, UploadProds, InitiateTX, ExecuteTX, and DeliverProds. Table II lists the symbols used in this work. These symbols are also used in later sections to represent the scheme.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

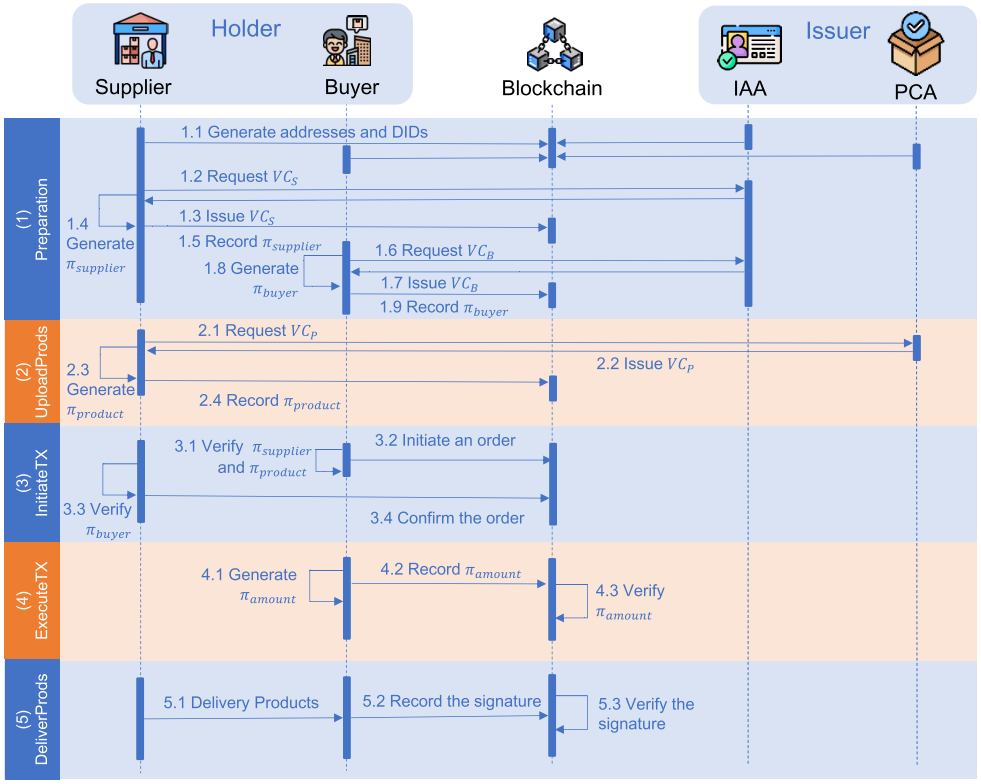


Fig. 1. Sequence diagram of ZKVeil.

In the Preparation phase, participants register blockchain accounts, generate DIDs, and obtain VCs from a trusted IAA. Participants generate ZKPs to demonstrate their qualifications without revealing sensitive data. In the UploadProds phase, suppliers securely upload product specifications verified by PCA. ZKPs ensure the specifications meet industry standards without disclosing confidential product details. In the InitiateTX phase, buyers initiate purchase orders confidentially. Order details are encrypted and hashed to protect privacy and integrity. Suppliers confirm orders by verifying the integrity and authenticity of transactions. In the ExecuteTX phase, transaction amounts remain confidential and regulatory compliant. Leveraging the BlockMaze privacy-preserving payment mechanism, buyers create encrypted transaction commitments and ZKPs to prove transaction amounts meet regulatory thresholds. Funds are locked until buyers confirm successful delivery. In the DeliverProds phase, the supplier delivers the goods off-chain, and the buyer submits a signed delivery confirmation on-chain. Once verified, the smart contract finalizes the transaction and releases the payment to the supplier.

ZKVeil incorporates protocol-level optimizations. First, ZKVeil integrates authority-certified credential compliance into the zero-knowledge circuits, enabling on-chain proofs to establish authenticity and regulatory compliance without revealing sensitive information. Second, compliance verification is embedded into the transaction workflow, ensuring that transactions proceed only after on-chain policy checks are successfully completed. Third, ZKVeil adopts a delivery-dependent payment locking with timeout mechanism, ensuring that payment settlement is atomically coupled with delivery confirmation or timeout-based release. The following sections detail the formal construction of each phase.

B. Construction of ZKVeil

1) Preparation Phase: Participants $\mathcal{P} = \{p_1, p_2, \ldots, p_n\}$ in the supply chain need to generate their accounts on the blockchain. The blockchain accounts $\mathcal{A} = \{a_1, a_2, \ldots, a_n\}$ are used to execute transactions and interact with the smart contract. Each account a_i is defined as: $a_i = (addr_i, pk_i, sk_i)$, where $addr_i$ is the blockchain address, pk_i is the public key, and sk_i is the private key. Registration function $Reg : \mathcal{P} \to \mathcal{A}$, mapping participants to blockchain accounts.

Blockchain accounts lack structured identity information. Supply chain systems require identity attestations and verifiable credentials. Decentralized Identifiers (DIDs) $DIDs = \{did_1, did_2, \ldots, did_n\}$ are a type of globally unique identifier that enables verifiable, self-sovereign digital identity. Each DID structure is defined as: $did_i = (id_i, Doc_i)$, where id_i is the unique identifier and Doc_i is the DID document containing public keys, identity verification methods, and so on. The transition from blockchain accounts to DIDs is formalized as follows. Participants create blockchain accounts $p_i \xrightarrow{Reg} a_i$, where $p_i \in \mathcal{P}$ and $a_i \in \mathcal{A}$. Then, participants create DIDs $a_i \xrightarrow{GenDID} did_i$. The DID generation function $GenDID : \mathcal{A} \to DIDs$ maps blockchain accounts to DIDs.

Let $Infos$ represent the domain of identity and qualification information of the Supplier S. S send $Infos$ to IAA to verify the identity and qualifications. Upon successful verification, IAA issues verifiable credentials VC_S to S. Let VC_S represent the verifiable credential of the supplier S, which encodes

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

identity information such as business license, production capacity, and other necessary attributes. Each verifiable credential can be modeled as: $VC_S = \{Meta_S, Claims_S, Proof_S\}$, where $Meta_S$ includes credential metadata (issuer, issuance date, expiration date), $Claims_S$ represents the supplier's identity claims, and $Proof_S$ is the digital signature proving the integrity and authenticity of the $Claims_S$. For example, the quality certification center issues ISO 9001 certification to the supplier. It indicates that the supplier meets the requirements of the quality management system. The above credential issuance process ensures that the supplier qualifications are authentic, verifiable, and originate from a trusted authority, thereby fulfilling the data authenticity goal (G4) defined in our system model.

The supplier's qualification information may reveal the enterprise's production capacity, business scope, and market positioning, and competitors may use this information to develop more competitive strategies. Therefore, the supplier does not need to disclose the qualifications, but can prove that he possesses the relevant production qualification. There exists a zero-knowledge proof generation function such that:

$$\pi_{\text{supplier}} : \text{Prove}\left[\{qual_{a_i}\} \subseteq VC_S \wedge \{qual_{a_i}\} \models \Sigma\right],$$

where VC_S is the S's qualification credential, $qual_{a_i}$ is the qualification, which is required for the transaction and meets the regulatory policy Σ, and π_{supplier} is a zero-knowledge proof attesting to the supplier's qualifications. The supplier records the proof π_{supplier} to the blockchain BC as a transaction payload:$tx_S = Record(DID_S, \pi_{\text{supplier}})$.

Any party, including buyers or regulators, can verify the supplier's qualifications without accessing the underlying credential data using a verification function $Verify_{\text{supplier}}(\pi_{\text{supplier}}) \in \{0, 1\}$, where the output indicates whether π_{supplier} is valid or not. Consequently, the zero-knowledge proof mechanism simultaneously ensures *identity privacy* of suppliers by preventing the disclosure of sensitive qualification details, and *regulatory compliance* by enabling any party to publicly verify that the supplier satisfies the required qualifications (G1).

Similarly, since certain products may have regulatory constraints on the buyer's qualifications (e.g., age restrictions for purchasing tobacco or alcohol), it is necessary to verify that B's identity information satisfies the required transaction conditions.

Let $Info_B$ represent the identity and qualification information of the Buyer B. B sends $Info_B$ to IAA to verify the identity and qualifications. Upon successful verification, IAA issues verifiable credentials VC_B to B. Let VC_B represent the verifiable credential of the buyer B, which encodes identity information such as age, qualification, and others. Each verifiable credential can be modeled as: $VC_B = \{Meta_B, Claims_B, Proof_B\}$, where $Meta_B$ includes credential metadata (issuer, issuance date, expiration date), $Claims_B$ represents the buyer's identity claims (e.g., age, professional licenses), and $Proof_B$ is the digital signature proving the integrity and authenticity of the $Claims_B$. The process ensures that the buyer's identity and qualification data are authentic, verifiable, and originate from a trusted authority, thereby fulfilling the data authenticity goal (G4).

Let $\mathcal{T}_P$ be the qualification requirements for purchasing a specific product P. This may include constraints such as a minimum age, professional certifications, or geographical restrictions. The buyer B must prove that their identity attributes encoded in VC_B satisfy $\mathcal{T}_P$, i.e., $Claims_B \models \mathcal{T}_P$.

To ensure privacy, the buyer generates a zero-knowledge proof π_{buyer} that proves compliance with $\mathcal{T}_P$ without disclosing the specific values of $Claims_B$. The proof π_{buyer} can be formalized as:

$$\pi_{\text{buyer}} : \text{Prove}\left[Claims_B \models \mathcal{T}_P\right],$$

where $\mathcal{T}_P$ represents the regulatory constraints on buyer qualifications, and $Claims_B$ is the buyer's true identity claims in VC_B.

The buyer records the proof π_{buyer} to the blockchain BC along with their decentralized identifier DID_B:

$$tx_{\text{buyer}} = \text{Record}(DID_B, \pi_{\text{buyer}}).$$

Any party can verify the buyer's qualifications without accessing the underlying credential data using a verification function $Verify_{\text{buyer}}(\pi_{\text{buyer}}) \in \{0, 1\}$. Therefore, this design ensures *identity privacy* of buyer by preventing the disclosure of sensitive personal details, and *regulatory compliance* by verifying that the buyer identity attribute satisfies the requirements (G1).

2) The UploadProds Phase: After qualification verification of the supplier, the supplier S is required to upload product information to the blockchain. Let P_S represent the product information associated with S, and $Info_P$ denote the product specifications that describe product attributes such as material, weight, and size. The product information can be represented as:$Info_P = \{spec_1, spec_2, \ldots, spec_m\}$.

To protect the confidentiality of $Info_P$, the supplier S sends $Info_P$ to a trusted PCA. PCA carries out strict inspection on the specifications of the products to ensure that the product specifications are authentic (G4). Upon successful verification, the PCA issues a verifiable product credential VC_P to the supplier S. VC_P contains the product information $Info_P$ and is signed by the PCA. The supplier S receives the signed credential, which serves as proof that the product specifications are accurate. The verifiable product credential VC_P can be formally defined as: $VC_P = \{Meta_P, Claims_P, Proof_P\}$, where $Meta_P$ includes credential metadata (issuer, issuance date, expiration date), $Claims_P$ represents the product specifications, and $Proof_P$ is the digital signature proving the integrity and authenticity of the specifications. Here, $Info_P$ denotes the raw product specification data, while $Claims_P$ represents the corresponding specification claims embedded in the verifiable credential VC_P.

After ensuring the authenticity of product specifications by PCA, the supplier can generate a zero-knowledge proof that enables buyers to verify that the specifications conform to industry standards. To allow product verification without revealing sensitive information, the supplier S generates a zero-knowledge proof π_{product} to prove that certain product specifications $\{spec_{j_1}, spec_{j_2}, \ldots, spec_{j_k}\} \subseteq VC_P$ satisfy specific regulatory or industry standards Φ. This can be formalized as:

$$\pi_{\text{product}} : \text{Prove}\left[\{spec_{j_1}, spec_{j_2}, \ldots, spec_{j_k}\} \models \Phi\right],$$

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

where π_{product} represents a zero-knowledge proof attesting that the selected product specifications $\{spec_{j_1}, spec_{j_2}, \ldots, spec_{j_k}\}$ comply with the required policy Φ. For example, for food suppliers, the additive content in food requires strict testing by the PCA. There is no need to leak the data after testing, only to show that the content is in accordance with the regulations.

After generating the proof π_{product}, the supplier S uploads the proof and the corresponding product identifier PID_S to the blockchain BC. This process can be described as:

$$tx_{\text{product}} = \text{Record}(PID_S, \pi_{\text{product}}),$$

where PID_S uniquely identifies the product uploaded by S.

Other parties can retrieve and verify π_{product} from the blockchain using the verification function $Verify_{\text{product}}$, ensuring that the uploaded product specifications comply with the policy Φ without revealing the underlying product credential VC_P. The verification process can be expressed as $Verify_{\text{product}}(\pi_{\text{product}}) \in \{0, 1\}$. This verification mechanism ensures regulatory compliance of the uploaded product information while preserving confidentiality (G2).

In this phase, the supplier uploads product specifications to the blockchain. The specifications are verified by PCA and protected by ZKPs. This ensures that sensitive product information remains confidential while allowing buyers to verify compliance with industry standards.

3) The InitiateTX Phase: In this phase, the buyer B initiates a request to purchase products from the supplier S.

After establishing qualification, the buyer constructs an order O to initiate the transaction process. The order contains essential product information (e.g., product ID, quantity). To protect sensitive order details, the order is encrypted. Let $O = (O_{id}, PID_S, q, Info_{\text{D}}, Info_{\text{pay}}, \mathsf{status})$ represent the order created by the buyer B, with O_{id} being the unique order identifier, PID_S the product identifier, $Info_{\text{D}}$ the delivery information (e.g., address), $Info_{\text{pay}}$ the payment information (e.g., payment method, amount), and status the order status, which is set to $\mathsf{status} := \texttt{created}$. Since O may contain sensitive information (e.g., product type, quantity), it is encrypted before submission to the blockchain.

Let Enc be a public-key encryption function, where: $O^{enc} = Enc_{pk_S}(O)$, and pk_S is the supplier's public key. This ensures that only the intended recipient (supplier S) can decrypt and access the order details.

To prevent order tampering, B computes the hash of the order: $H(O) = \text{Hash}(O)$, and submits the hash along with the encrypted order O^{enc}, the proof π_{buyer}, and their decentralized identifier DID_B to the blockchain BC:

$$tx_{\text{order}} = \text{Record}(DID_B, H(O), O^{enc}, \pi_{\text{buyer}}).$$

Once the order tx_{order} is published, relevant parties (e.g., suppliers, regulators) can verify it using $Verify_{\text{buyer}}$ to confirm that the buyer possesses the required qualifications for purchasing the product P without accessing sensitive personal information, thereby enhancing privacy and ensuring compliance with regulatory policies.

The supplier uses its private key sk_S to decrypt the encrypted order: $O = Dec_{sk_S}(O^{enc})$. This ensures that only the intended supplier S can access the order details, maintaining the confidentiality of the order.

To ensure the order has not been tampered with during transmission, S computes the hash of the decrypted order: $H'(O) = \text{Hash}(O)$, and compares it with the order hash $H(O)$ submitted by the buyer: $H'(O) \stackrel{?}{=} H(O)$. If the hash values match, the supplier confirms that the order O has not been altered. After verifying the integrity of the order, the supplier processes the order O. Once the supplier S agrees to process the order O, it must confirm this agreement by publishing an order confirmation transaction on the blockchain. This confirmation transaction is shown as follows:

$$tx_{\text{confirm}} = \text{Record}(DID_S, H(O), Sig_S(O), \mathsf{status}).$$

The transaction tx_{confirm} includes the supplier's decentralized identifier DID_S, the hash of the order $H(O)$, the digital signature $Sig_S(O)$, and the order status status. The order status is set to $\mathsf{status} := \texttt{accepted}$, which indicates that the order has been accepted and is awaiting further processing.

In this phase, the buyer initiates a purchase order to the supplier. The order details are encrypted and hashed to protect privacy and integrity. The supplier confirms the order by verifying the integrity and authenticity of the transaction.

4) The ExecuteTX Phase: When buyer B initiates a payment to supplier S, the transfer relationship between them and the actual transaction amount v must remain confidential. ZKVeil chooses BlockMaze to implement the transfer of the amount. BlockMaze is a privacy-preserving payment system that allows users to transfer funds without revealing the transaction amount and the identities of the parties involved. ZKVeil adds a locking mechanism and verification of amount compliance based on BlockMaze.

First, the buyer B executes the mint operation BlockMaze.Mint to convert a plaintext balance $value_B \in \mathbb{Z}^+$ associated with account $addr_B$ into a private balance $\mathsf{zk\_balance}_B := (cmt_B, addr_B, value_B, sn_B, r_B)$, where $cmt_B := \mathsf{COMM}_{bc}(addr_B, addr_B, value_B, sn_B, r_B)$, sn_B is a serial number associated with cmt_B, and r_B is a random number masking sn_B.

Next, the buyer creates a fund transfer commitment $cmt_v := \mathsf{COMM}_{tc}(addr_B, v, pk_S, r_v, sn_B)$, where v is the transaction amount, pk_S is the public key of the supplier, r_v is a random number masking cmt_v, sn_B is a serial number. To ensure regulatory compliance, B generates a zero-knowledge proof π_{amt} showing that:

$$\pi_{\text{amt}} : \mathsf{Proof}\left[0 \le v \le \Theta\right].$$

This proof demonstrates that v prevents negative payments, and does not exceed the regulatory threshold Θ. Importantly, the proof reveals nothing about the specific value of v, beyond the fact that it lies within the specified range (G3).

The tuple $(cmt_v, \pi_{\text{amt}})$ is submitted to the blockchain through BlockMaze.Send. Any party on the blockchain can check the validity of the proof $Verify_{\text{amt}}(\pi_{\text{amt}}) \in \{0, 1\}$. The verification function $Verify_{\text{amt}}$ returns 1 if and only if the proof π_{amt} is valid and the committed value v meets the regulatory constraint $0 \le v \le \Theta$. This verification occurs entirely on-chain, requiring no trusted third party, and preserves the confidentiality of the transaction amount.

After amount verification, the supplier S collects multiple commitments $\{cmt_v^{(i)}\}$, constructs a Merkle tree $MT_T = \mathsf{MerkleTree}(\{cmt_v^{(i)}\})$. S executes BlockMaze.Deposit to prove

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

that its cmt_v exists on MT_T and get cmt_v. Then, the contract sets status := pending. The supplier S waits for the transaction to be confirmed by the buyer B.

In this phase, the buyer initiates a payment to the supplier while ensuring that the transaction amount remains confidential. The buyer generates a fund transfer commitment and a zero-knowledge proof to demonstrate compliance with regulatory constraints.

5) The DeliveryProds Phase: S delivers the agreed product to B, which occurs off-chain. Let $Info_{\mathrm{D}}$ denote the private delivery-related information, including the delivery ID, timestamp, address, etc. The buyer encrypts $Info_{\mathrm{D}}$ using the supplier's public key pk_S, resulting in:

$$\mathcal{E}_{pk_S}(D) = \mathsf{Enc}_{pk_S}(Info_{\mathrm{D}}).$$

This ensures that only the supplier can decrypt and access the delivery details.

To confirm the authenticity of the submission, the buyer also signs the hash of the encrypted delivery data using their private key sk_B. Let $\sigma_B = \mathsf{Sign}_{sk_B}(H(\mathcal{E}_{pk_S}(D)))$ denote the buyer's signature, where $H(\cdot)$ is a cryptographic hash function. This signature provides non-repudiation, ensuring that only the legitimate buyer can acknowledge receipt.

The buyer B then submits the following to the blockchain:

$$tx_{\mathrm{delivery}} = \mathsf{Record}(DID_B, \mathcal{E}_{pk_S}(D), \sigma_B).$$

Upon receiving the delivery confirmation transaction tx_{delivery}, the smart contract verifies the buyer's signature:

$$\mathsf{Verify}_{\sigma}(\sigma_B, H(\mathcal{E}_{pk_S}(D)), pk_B) = \begin{cases} 1 & \text{if valid signature,} \\ 0 & \text{otherwise.} \end{cases}$$

If the signature is valid, the smart contract updates the transaction status to status := completed. Finally, S runs BlockMaze.Redeem to convert cmt_v into plaintext balance.

To address the risk that a malicious buyer may receive the products but refuse to submit the confirmation signature σ_B, we introduce a timeout mechanism. Let T_{ack} be the delivery acknowledgment deadline. If no valid signature is received before T_{ack}, the smart contract allows the supplier S to invoke a timeout-based fallback. This ensures that payment is eventually released if the buyer remains unresponsive beyond a reasonable delivery window.

In this phase, the buyer confirms delivery by encrypting the delivery details and signing the confirmation. Upon successful verification (either by signature or timeout), the smart contract releases the funds to the supplier.

VI. Theoretical Security Analysis

In this section, we provide a formal security analysis of ZKVeil using the simulation-based security paradigm. We establish security under standard cryptographic assumptions and provide detailed proofs with complete derivations.

A. Security Assumptions

The security of ZKVeil relies on the following standard assumptions:

(A1) ZK-SNARK Soundness: No PPT adversary can produce a valid proof for a false statement except with negligible probability.

(A2) ZK-SNARK Zero-Knowledge: The proof reveals no information about the witness beyond the validity of the statement.

(A3) Commitment Binding and Hiding: The Pedersen commitment scheme is computationally binding under the discrete logarithm assumption and perfectly hiding.

(A4) IND-CPA Security: The public-key encryption scheme is indistinguishable under chosen-plaintext attacks (IND-CPA).

(A5) EUF-CMA Security: The digital signature scheme is existentially unforgeable under chosen-message attacks (EUF-CMA).

B. Ideal Functionality $\mathcal{F}_{ZKVeil}$

The ideal functionality $\mathcal{F}_{\mathrm{ZKVeil}}$ is formally specified in Fig. 2. Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary who may statically corrupt at most one party, either the supplier S^* or the buyer B^*. The functionality is defined as a state machine with the following components.

1) Parameters: A protocol execution may involve multiple concurrent sessions, each identified by a session identifier $sid = (DID_B, DID_S, ts)$, where DID_B and DID_S are the decentralized identifiers (DIDs) of the buyer and supplier, and ts is a session-specific timestamp/nonce used to prevent replay across sessions. OID is an order identifier and PID is a product identifier. $(\Sigma, \mathcal{T}_P, \Phi, \Theta)$ are regulatory policies. Let $DID_B(sid)$ and $DID_S(sid)$ denote the buyer/supplier DID components embedded in sid.

The current time is now and a deadline $deadline$ = now + T_{ack} is set when payment is locked. A *timeout event* for session sid occurs when now > $deadline$. T_{ack} is timeout parameter.

It maintains two authorization flags $regB[sid], regS[sid] \in \{0, 1\}$, initially 0. We interpret $st[sid]$ = REG as that both parties have been authorized, i.e., $regB[sid] = regS[sid] = 1$. It also maintains a product binding record $prodOwner[PID] \in DIDs \cup \{\perp\}$, initialized to $\perp$, a per-session product record $prodPID[sid] \in \{\perp\} \cup \{PID\}$, initialized to $\perp$, and an order record $ord[sid] = (OID, h_O, O^{enc})$ or $\perp$, where h_O is a public order hash (e.g., $h_O = H(O)$) and O^{enc} denotes the publicly recorded encrypted order.

2) State: For each session $sid = (DID_B, DID_S, ts)$, the functionality maintains a state variable

$$st[sid] \in \{\mathsf{INIT}, \mathsf{REG}, \mathsf{PROD}, \mathsf{ORDER}, \mathsf{LOCKED}, \mathsf{DONE}\},$$

and an escrow record $lock[sid] = (cmt, deadline)$ or $\perp$, where $\perp$ denotes an empty value. $lock[sid]$ abstracts a pending payment that has been verified as compliant and locked, but not yet released to the supplier, together with its associated timeout. Different sessions identified by distinct sid are handled independently. For each session identified by sid, the ideal functionality $\mathcal{F}_{\mathrm{ZKVeil}}$ maintains a finite-state machine $st[sid]$ capturing the abstract execution progress. Each state has the following precise semantics: INIT denotes that the session has been created but no participant has been authorized; REG denotes that the buyer and supplier have been successfully authorized with respect to the corresponding policies; PROD denotes that a product associated with the session has been verified as compliant; ORDER denotes that a valid order has been initiated by the buyer; LOCKED denotes that a compliant payment commitment has been escrowed and the payment

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

Ideal Functionality $\mathcal{F}_{\text{ZKVeil}}$

(1) Register. Upon receiving $(\mathsf{REGISTER}, sid, DID_i, role_i)$ from party P_i:

- If $st[sid] \neq \mathsf{INIT}$ or $role_i \notin \{\mathsf{SUPPLIER}, \mathsf{BUYER}\}$, ignore the request.
- Compute $b_1 \leftarrow \mathsf{AuthQuery}(DID_i, role_i)$, where $b_1 \in \{0,1\}$ indicates whether P_i is authorized with respect to policy Σ (supplier) and $\mathcal{T}_P$ (buyer).
- If $b_1 = 1$, then:
  - If $role_i = \mathsf{BUYER}$, $regB[sid] \leftarrow 1$.
  - If $role_i = \mathsf{SUPPLIER}$, $regS[sid] \leftarrow 1$.
  - If $regB[sid] = regS[sid] = 1$, $st[sid] \leftarrow \mathsf{REG}$.

  and output $(\mathsf{REGISTER\_OK}, DID_i)$ to P_i; otherwise output $(\mathsf{REGISTER\_FAIL}, DID_i)$.
- Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{REGISTER}, sid, DID_i, role_i, b_1)$.

---

(2) Upload Product. Upon receiving $(\mathsf{UPLOAD}, sid, DID_S, PID)$ from supplier S:

- If $st[sid] \neq \mathsf{REG}$, output $(\mathsf{UPLOAD\_FAIL}, PID)$ to S.
- If $DID_S \neq DID_S(sid)$, output $(\mathsf{UPLOAD\_FAIL}, PID)$ to S.
- If $prodOwner[PID] = \perp$, $prodOwner[PID] \leftarrow DID_S$; otherwise, if $prodOwner[PID] \neq DID_S$, output $(\mathsf{UPLOAD\_FAIL}, PID)$ to S.
- Compute $b_2 \leftarrow \mathsf{SpecQuery}(PID)$, where $b_2 \in \{0,1\}$ indicates whether the product satisfies specification policy Φ.
- If $b_2 = 1$, $st[sid] \leftarrow \mathsf{PROD}$, $prodPID[sid] \leftarrow PID$, and output $(\mathsf{UPLOAD\_OK}, PID)$ to S; otherwise output $(\mathsf{UPLOAD\_FAIL}, PID)$.
- Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{UPLOAD}, sid, DID_S, PID, b_2)$.

---

(3) Initiate Order. Upon receiving $(\mathsf{ORDER}, sid, h_O, O^{enc})$ from buyer B:

- If $st[sid] \neq \mathsf{PROD}$, output $(\mathsf{ORDER\_FAIL}, \perp)$ to B.
- Sample a fresh order identifier $OID \leftarrow \{0,1\}^\lambda$. Let $ord[sid] \leftarrow (OID, h_O, O^{enc})$ and $st[sid] \leftarrow \mathsf{ORDER}$.
- Output $(\mathsf{ORDER\_OK}, OID)$ to both B and S.
- Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{ORDER}, sid, OID, h_O, O^{enc})$.

---

(4) Lock Payment. Upon receiving $(\mathsf{LOCK}, sid, cmt)$ from buyer B:

- If $st[sid] \neq \mathsf{ORDER}$ or $ord[sid] = \perp$, output $(\mathsf{LOCK\_FAIL}, \perp)$ to B.
- Compute $b_3 \leftarrow \mathsf{AmtQuery}(sid, cmt)$, where $b_3 \in \{0,1\}$ indicates whether the committed amount satisfies the threshold Θ.
- If $b_3 = 1$, $st[sid] \leftarrow \mathsf{LOCKED}$, let $lock[sid] \leftarrow (cmt, \mathsf{now} + T_{\mathsf{ack}})$, and output $(\mathsf{PAY\_LOCKED}, OID)$ to both B and S; otherwise output $(\mathsf{LOCK\_FAIL}, OID)$ to B.
- Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{LOCK}, sid, cmt, b_3)$.

---

(5) Confirm Delivery or Timeout.

- Upon receiving $(\mathsf{CONFIRM}, sid, h_D)$ from buyer B, if $st[sid] = \mathsf{LOCKED}$, $st[sid] \leftarrow \mathsf{DONE}$ and output $(\mathsf{COMPLETE}, \perp)$ to both B and S. Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{DONE}, sid, \mathsf{CONFIRM}, h_D)$.
- Upon a timeout event for any sid with $st[sid] = \mathsf{LOCKED}$ and $lock[sid] = (cmt, deadline)$ and $\mathsf{now} > deadline$, let $st[sid] \leftarrow \mathsf{DONE}$ and output $(\mathsf{COMPLETE\_TIMEOUT}, \perp)$ to S. Leak to $\mathcal{A}$: $(\mathsf{LEAK}, \mathsf{DONE}, sid, \mathsf{TIMEOUT}, \perp)$.

Security-Relevant Leakage. The adversary learns only session identifiers, DIDs, product IDs, order identifiers, payment commitments, and pass/fail bits of policy checks. It never learns identity credentials, product specifications, transaction amounts, or delivery information.

Fig. 2. Ideal functionality $\mathcal{F}_{\text{ZKVeil}}$.

is locked; and DONE denotes that the transaction has been completed, either by buyer confirmation or by timeout.

3) Operations: The commands in $\mathcal{F}_{\text{ZKVeil}}$ are abstract operations capturing the intended security semantics rather than concrete messages. REGISTER denotes the registration of participants' identity; UPLOAD denotes the uploading of product information; ORDER denotes initiation of transactions; LOCK denotes escrow of a payment commitment; and CONFIRM denotes delivery confirmation that finalizes the transaction. All information observable by the adversary is explicitly released via LEAK outputs. The adversary $\mathcal{A}$ is PPT and receives only explicit leakage of the form $(\mathsf{LEAK}, \cdot)$.

For compactness, policy checks are abstracted as verification procedures that output a bit $b \in \{0,1\}$: AuthQuery (identity compliance), SpecQuery (specification compliance), and AmtQuery (amount compliance). Concretely, these procedures are executed by $\mathcal{F}_{\text{ZKVeil}}$ itself and are defined to be consistent with the corresponding public verification results in the real protocol. The adversary $\mathcal{A}$ learns only the explicit leakage of the form $(\mathsf{LEAK}, \cdot)$ released by $\mathcal{F}_{\text{ZKVeil}}$ and does not choose the outcomes of these checks.

C. Simulation-Based Security Definition

Definition 1 (Real-World Execution): The real-world execution $\mathsf{REAL}_{\Pi,\mathcal{A}}(1^\lambda)$ is defined as the execution of protocol Π_{ZKVeil} in the presence of a PPT adversary $\mathcal{A}$, who controls the corrupted party, observes all public blockchain messages, and receives the outputs of corrupted parties. The experiment outputs the bit produced by a PPT distinguisher given the adversary's view and the public transcript.

Definition 2 (Ideal-World Execution): The ideal-world execution $\mathsf{IDEAL}_{\mathcal{F},\mathcal{S}}(1^\lambda)$ is defined as the execution where all parties interact only with the ideal functionality $\mathcal{F}_{\text{ZKVeil}}$. A PPT simulator $\mathcal{S}$ generates a simulated adversarial view using only the information explicitly leaked by $\mathcal{F}_{\text{ZKVeil}}$. The experiment outputs the bit produced by a PPT distinguisher given the simulated view and the induced public transcript.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

Definition 3 (Secure Realization): Protocol Π_{ZKVeil} securely realizes $\mathcal{F}_{\text{ZKVeil}}$ if for every PPT adversary $\mathcal{A}$ corrupting at most one party (buyer or supplier), there exists a PPT simulator $\mathcal{S}$ such that

$$\mathsf{REAL}_{\Pi,\mathcal{A}}(1^\lambda) \approx_c \mathsf{IDEAL}_{\mathcal{F},\mathcal{S}}(1^\lambda),$$

where 1^λ is the security parameter in unary representation.

D. Security Proof

Theorem 1 (Security of ZKVeil): Under assumptions (A1)–(A5), Protocol Π_{ZKVeil} securely realizes the ideal functionality $\mathcal{F}_{\text{ZKVeil}}$ in the presence of a PPT adversary corrupting at most one party (buyer or supplier).

Proof: We prove the theorem using the real-world/ideal-world simulation paradigm. For each possible corruption pattern, we explicitly construct a PPT simulator whose output is computationally indistinguishable from the adversary's view in the real execution for the same session *sid*, consistent with the explicit leakage of $\mathcal{F}_{\text{ZKVeil}}$.

1) Case 1 (Corrupted Buyer B^*): We consider the case where the adversary $\mathcal{A}$ statically corrupts the buyer B^*. The adversary $\mathcal{A}$ obtains the full internal state of B^* and controls all messages sent on its behalf, while observing the blockchain transcript. The supplier S, IAA, and PCA are honest. We show that Protocol Π_{ZKVeil} securely realizes $\mathcal{F}_{\text{ZKVeil}}$ in this setting by constructing a PPT simulator $\mathcal{S}_B$.

a) Simulator $\mathcal{S}_B$: The simulator $\mathcal{S}_B$ internally runs $\mathcal{A}$ and emulates the blockchain. All simulated on-chain artifacts are generated as needed to match the leakage and interface prescribed by $\mathcal{F}_{\text{ZKVeil}}$ for each session *sid*. It is logically driven by the inputs and outputs of $\mathcal{F}_{\text{ZKVeil}}$. For readability, we describe $\mathcal{S}_B$ according to the phases of the real protocol.

At the beginning of the simulation, $\mathcal{S}_B$ samples fresh key pairs (pk_S, sk_S) for the honest supplier S, and registers/publishes the corresponding public keys on the blockchain. The secret key sk_S is kept internally by $\mathcal{S}_B$ solely for producing supplier-side ciphertext decryptions and signatures consistent with the public transcript.

Register. When $\mathcal{A}$ (on behalf of B^*) submits a registration, $\mathcal{S}_B$ forwards $(\mathsf{REGISTER}, sid, DID_B, \mathsf{BUYER})$ to $\mathcal{F}_{\text{ZKVeil}}$ and relays the response $(\mathsf{REGISTER\_OK}, DID_B)$ or $(\mathsf{REGISTER\_FAIL}, DID_B)$ to $\mathcal{A}$. For the honest supplier S, $\mathcal{S}_B$ initiates $(\mathsf{REGISTER}, sid, DID_S, \mathsf{SUPPLIER})$ and publishes the corresponding public registration information.

Upload Product. The simulator $\mathcal{S}_B$ generates simulated proofs for the corresponding public statements using the zero-knowledge property of the proof system. By Assumption (A2), these simulated proofs are computationally indistinguishable from honestly generated proofs. Upon receiving $(\mathsf{UPLOAD}, sid, PID)$ from the honest supplier, $\mathcal{S}_B$ forwards it to $\mathcal{F}_{\text{ZKVeil}}$ and emulates the on-chain outcome consistent with the returned bit.

Initiate Order. When $\mathcal{A}$ submits an order transaction $\mathsf{Record}(DID_B, H(O), O^{enc}, \pi_{\mathsf{buyer}})$, $\mathcal{S}_B$ performs the same public verification as the real smart contract. If verification succeeds, $\mathcal{S}_B$ sends $(\mathsf{ORDER}, sid, h_O, O^{enc})$ to $\mathcal{F}_{\text{ZKVeil}}$. Since $\mathcal{S}_B$ emulates the honest supplier and holds sk_S, $\mathcal{S}_B$ decrypts O^{enc} using sk_S and returns a confirmation transaction containing a valid signature $\mathsf{Sig}_{sk_S}(O)$. This distribution is identical to the real execution.

Lock Payment. When $\mathcal{A}$ submits a payment commitment $(cmt_v, \pi_{\mathsf{amt}})$, $\mathcal{S}_B$ runs the public verifier $\mathsf{Verify}_{\mathsf{amt}}(\pi_{\mathsf{amt}})$. If the verification is successful, it forwards $(\mathsf{LOCK}, sid, cmt_v)$ to $\mathcal{F}_{\text{ZKVeil}}$; otherwise, it ensures the session does not enter the locked state. The simulator $\mathcal{S}_B$ locally evaluates $\mathsf{AmtQuery}(sid, cmt_v)$ using the same public verification procedure as in the real protocol and ensures that the resulting ideal-world transition matches the real-world outcome.

Confirm Delivery or Timeout. If $\mathcal{A}$ submits a delivery confirmation, $\mathcal{S}_B$ forwards $(\mathsf{CONFIRM}, sid, h_D)$ to $\mathcal{F}_{\text{ZKVeil}}$ whenever the session is in state LOCKED. h_D is the public commitment on-chain for delivery confirmation. Otherwise, $\mathcal{S}_B$ lets $\mathcal{F}_{\text{ZKVeil}}$ complete the session via its timeout rule and emulates the corresponding public completion event.

b) Hybrid argument: We prove the indistinguishability between the real and ideal executions via a sequence of hybrid experiments.

Hybrid H_0 (Real execution). This hybrid corresponds to the real-world execution of Π_{ZKVeil} with a corrupted buyer B^*, where the honest supplier generates all ZKPs and protocol messages according to the specification.

Hybrid H_1 (Simulated supplier qualification proofs). We modify H_0 by replacing the honest supplier's qualification proof π_{supplier} with a simulated proof $\tilde{\pi}_{\mathsf{supplier}}$ for the same public statement, generated using the zero-knowledge property of the proof system. By Assumption (A2), H_1 is computationally indistinguishable from H_0.

Hybrid H_2 (Simulated product compliance proofs). Starting from H_1, we further replace the honest supplier's product compliance proof π_{product} with a simulated proof $\tilde{\pi}_{\mathsf{product}}$ for the same public statement. Again, by Assumption (A2), H_2 is computationally indistinguishable from H_1.

Hybrid H_3 (Ideal functionality-driven execution). Finally, we replace the remaining honest supplier behavior by interactions driven by the ideal functionality $\mathcal{F}_{\text{ZKVeil}}$ and transcript emulation by the simulator $\mathcal{S}_B$. In this hybrid, state transitions, acceptance/rejection outcomes, and timeout events are determined solely by $\mathcal{F}_{\text{ZKVeil}}$, while $\mathcal{S}_B$ generates a blockchain transcript consistent with the explicit leakage of $\mathcal{F}_{\text{ZKVeil}}$.

In particular, any supplier-side proofs appearing on-chain are generated as in H_2, and any ciphertexts or commitments appearing in the public transcript are distributed indistinguishably from the real execution under the Assumption (A4). Therefore, the adversary's view in H_3 is computationally indistinguishable from its view in H_2. Combining the above hybrids, we conclude that the real and ideal executions are computationally indistinguishable:

$$\mathsf{REAL}_{\Pi_{\text{ZKVeil}},\mathcal{A}} \approx_c \mathsf{IDEAL}_{\mathcal{F}_{\text{ZKVeil}},\mathcal{S}_B}.$$

2) Case 2 (Corrupted Supplier S^*): We consider the case where the adversary $\mathcal{A}$ statically corrupts the supplier S^*. The adversary $\mathcal{A}$ obtains the full internal state of S^* and controls all messages sent on its behalf, while observing the blockchain transcript. The buyer B, IAA, and PCA are honest.

a) Simulator $\mathcal{S}_S$: The simulator $\mathcal{S}_S$ internally runs $\mathcal{A}$ and emulates the blockchain. The goal of $\mathcal{S}_S$ is to generate a view for $\mathcal{A}$ that is computationally indistinguishable from its view in a real execution, while interacting only with the ideal functionality $\mathcal{F}_{\text{ZKVeil}}$. For readability, we describe $\mathcal{S}_S$ according to the phases of the real protocol.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

Register. When $\mathcal{A}$ (on behalf of S^*) submits a registration, $\mathcal{S}_S$ forwards $(\mathsf{REGISTER}, sid, DID_S, \mathsf{SUPPLIER})$ to $\mathcal{F}_{\mathrm{ZKVeil}}$ and relays the response $(\mathsf{REGISTER\_OK}, DID_S)$ or $(\mathsf{REGISTER\_FAIL}, DID_S)$ to $\mathcal{A}$. For the honest buyer B, $\mathcal{S}_S$ initiates $(\mathsf{REGISTER}, sid, DID_B, \mathsf{BUYER})$ and publishes the corresponding public registration information.

Upload Product. Since the supplier S^* is corrupted, all supplier-side ZKPs and product upload messages are generated by $\mathcal{A}$ itself. The simulator only checks whether the submitted proofs pass the public verification algorithms. Upon receiving $(\mathsf{UPLOAD}, sid, PID)$ from $\mathcal{A}$, $\mathcal{S}_S$ forwards it to $\mathcal{F}_{\mathrm{ZKVeil}}$ and emulates the on-chain outcome consistent with the returned bit from the specification query. No additional information about product specifications beyond the pass/fail bit is revealed.

Initiate Order. When the honest buyer submits an order transaction $\mathsf{Record}(DID_B, H(O), O^{enc}, \pi_{\mathsf{buyer}})$, $\mathcal{S}_S$ emulates the blockchain behavior by publishing the encrypted order and hash, and forwards $(\mathsf{ORDER}, sid, h_O, O^{enc})$ to $\mathcal{F}_{\mathrm{ZKVeil}}$. If the corrupted supplier responds with an order confirmation transaction containing a signature $\mathsf{Sig}_{S^*}(O)$, $\mathcal{S}_S$ performs the same public signature verification as the real smart contract and emulates the corresponding acceptance or rejection outcome.

Lock Payment. When the honest buyer submits a payment commitment $(cmt_v, \pi_{\mathsf{amt}})$, $\mathcal{S}_S$ runs the public verifier $\mathsf{Verify}_{\mathsf{amt}}(\pi_{\mathsf{amt}})$. If the verification is successful, $\mathcal{S}_S$ forwards $(\mathsf{LOCK}, sid, cmt_v)$ to $\mathcal{F}_{\mathrm{ZKVeil}}$; otherwise, it ensures that the session does not enter the locked state. The simulator $\mathcal{S}_S$ locally computes $b_3 \leftarrow \mathsf{AmtQuery}(sid, cmt_v)$ based on the same on-chain/public verification logic as in the real execution, and ensures that the state transition of $\mathcal{F}_{\mathrm{ZKVeil}}$ is consistent with this verification result.

Confirm Delivery or Timeout. If the honest buyer submits a delivery confirmation, $\mathcal{S}_S$ forwards $(\mathsf{CONFIRM}, sid, h_D)$ to $\mathcal{F}_{\mathrm{ZKVeil}}$ whenever the session is in state LOCKED. Otherwise, if no valid confirm message is delivered before the ideal-world timeout deadline maintained by $\mathcal{F}_{\mathrm{ZKVeil}}$, $\mathcal{S}_S$ lets $\mathcal{F}_{\mathrm{ZKVeil}}$ complete the session via its timeout rule and emulates the corresponding public completion event.

b) Hybrid argument: We prove the indistinguishability between the real and ideal executions via a sequence of hybrid experiments.

Hybrid H_0 *(Real execution).* This hybrid corresponds to the real-world execution of Π_{ZKVeil} with a corrupted supplier S^*, where the honest buyer generates all ZKPs and protocol messages according to the specification.

Hybrid H_1 *(Simulated buyer qualification proofs).* We modify H_0 by replacing the honest buyer's qualification proof π_{buyer} with a simulated proof $\tilde{\pi}_{\mathsf{buyer}}$ for the same public statement, generated using the zero-knowledge property of the proof system. By the Assumption (A2), H_1 is computationally indistinguishable from H_0.

Hybrid H_2 *(Simulated amount commitments).* Starting from H_1, we further replace the honest buyer's payment-locking tuple $(cmt_v, \pi_{\mathsf{amt}})$ with a simulated tuple $(\tilde{cmt}, \tilde{\pi}_{\mathsf{amt}})$ generated as follows: the simulator samples an arbitrary dummy amount $\tilde{v} \in [0, \Theta]$ and fresh randomness $\tilde{r}$, computes $\tilde{cmt} := \mathsf{COMM}_{tc}(addr_B, \tilde{v}, pk_S, \tilde{r}, sn_B)$, and produces a proof $\tilde{\pi}_{\mathsf{amt}}$ for the public statement that the committed value lies in the range $[0, \Theta]$. By the hiding of the Pedersen commitment (Assumption (A3)), the distribution of $\tilde{cmt}$ is identical to that of cmt_v even though $\tilde{v}$ is independent of the real amount. Moreover, by the Assumption (A2), the proof $\tilde{\pi}_{\mathsf{amt}}$ reveals no information about $\tilde{v}$ beyond validity. Therefore, H_2 is computationally indistinguishable from H_1.

Hybrid H_3 *(Ideal functionality-driven execution).* Finally, we replace the remaining honest buyer behavior by interactions driven by the ideal functionality $\mathcal{F}_{\mathrm{ZKVeil}}$ and transcript emulation by the simulator $\mathcal{S}_S$. In this hybrid, state transitions, acceptance or rejection outcomes, and timeout events are determined solely by $\mathcal{F}_{\mathrm{ZKVeil}}$, while $\mathcal{S}_S$ generates the blockchain transcript consistent with the explicit leakage of $\mathcal{F}_{\mathrm{ZKVeil}}$.

All values observable to the adversary in H_2 are identically distributed to those in H_1. Moreover, the encrypted order O^{enc} and any other ciphertexts generated by the honest buyer are computationally indistinguishable from those in the real execution under the Assumption (A4). In addition, any payment commitments posted on-chain (e.g., cmt_v) leak no information about the underlying amount due to the Assumption (A3). Hidden information such as buyer credentials, transaction amounts, and delivery details never appear in the transcript. Therefore, H_3 is computationally indistinguishable from H_2. Combining the above hybrids, we conclude that

$$\mathsf{REAL}_{\Pi_{\mathrm{ZKVeil}},\mathcal{A}} \approx_c \mathsf{IDEAL}_{\mathcal{F}_{\mathrm{ZKVeil}},\mathcal{S}_S}.$$

We additionally show that, except with negligible probability, verification correctness and protocol execution consistency are preserved. Specifically, any adversarial attempt to make the verification procedure accept an invalid statement, such as a false qualification claim, would imply a valid ZKP for a false statement, contradicting Assumption (A1). Similarly, forging an acknowledgment signature on behalf of an honest party would violate Assumption (A5). Therefore, the accept/reject outcomes and transaction semantics in the real execution coincide with those defined by the ideal functionality $\mathcal{F}_{\mathrm{ZKVeil}}$.■

VII. Performance Evaluation

In this section, we will evaluate the performance of ZKVeil by experiments.

A. Experimental Setup

All experiments are executed within a Linux-based Docker container deployed on a physical workstation with two Intel Xeon Gold 6226R CPUs (2.90 GHz), 128 GB RAM, and a mixed storage configuration consisting of a 954 GB NVMe SSD and a 3.64 TB HDD. The host operating system is Windows 11 (64-bit), which serves only as the container host and development interface, while all experimental components run entirely inside the Linux container.

The Ethereum client used is Geth (v1.13.15),[4] which facilitates the creation and management of Ethereum nodes. A private Ethereum network comprising 100 nodes is established, with one main node responsible for generating the genesis block and configuring network parameters, while the remaining 99 nodes function as participants. We use Clique consensus to ensure that all nodes can reach consensus on the blockchain state. The gas limit is set to 8 million gas per block.

[4]https://geth.ethereum.org/

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

We use Hardhat (v2.23.0)[5] as the development framework for smart contracts, enabling efficient contract deployment and testing. The Solidity compiler version is set to 0.8.28, ensuring compatibility with the latest language features and optimizations.

The zero-knowledge proof system is implemented with ZoKrates,[6] a zk-SNARK toolbox for Ethereum, utilizing the Groth16 scheme for ZKP generation and verification.

We utilize Ethr-did[7] to generate decentralized identifiers (DIDs) for participants, ensuring secure and privacy-preserving identity management.

B. Application Example

We take the ship manufacturing supply chain as an example. The buyer is the ship owner, and the supplier is the ship builder. Regulation and privacy are mainly presented in three aspects.

Firstly, the supplier needs to verify the buyer's qualifications. The buyer must also ensure that the supplier meets the necessary qualifications for building ships. Taking the supplier as an example, the qualification list of the supplier is private information, while the target qualification to be verified is public information. It is necessary to verify whether the target qualification is included in the qualification list.

Secondly, the ship specifications include the ship's length, width, height, weight, and so on. The specifications are also private information, and the ship supplier needs to prove that the ship specifications meet the requirements without revealing the specific values. In the experiment, we use a support plate as an example. According to the Steel Ship Classification Rule,[8] each supporting device should be equipped with a support plate. The specifications of the plate include the ship width B, waterline depth d, and material factor K of the support plate. The height h, thickness t, area A of the supporting plate should meet the following requirements:

$$h \geq 42(B+d) - 70,$$
$$T = (0.01h + 3)\sqrt{K}, 6 \leq T \leq 12,$$
$$A \geq (4.8d - 3)K.$$

Thirdly, the amount of shipbuilding is also private information, and the amount of shipbuilding meets the required range without revealing the specific value.

C. Performance Results

We evaluate gas consumption, transactions per second (TPS), and the time and memory cost for proof generation and verification.

1) Gas Consumption: Gas consumption is a metric for measuring the computational cost of smart contract execution on Ethereum. We measure the gas consumption of core procedures in ZKVeil, including identity verification, specification verification, and amount verification. Table III summarizes the gas consumption for different verification functions in ZKVeil. The gas cost is calculated based on the average gas price, which is set at 1.8 Gwei. The cost in Ether and USD is also provided for reference, assuming an ETH/USD price of 3089.[9] The processes of verification consume a certain amount of gas, which is reasonable since they include complex computations such as bilinear pairing computations. The complexity of the verification function directly affects gas consumption and transaction costs.

TABLE III
GAS CONSUMPTION FOR VERIFICATION FUNCTIONS

Function	Gas Used	Ether Cost	USD
$\mathsf{Verify}(\pi_{\mathsf{buyer}})$	253,396	0.00045611	1.41
$\mathsf{Verify}(\pi_{\mathsf{supplier}})$	253,202	0.00045587	1.41
$\mathsf{Verify}(\pi_{\mathsf{product}})$	234,935	0.00042288	1.31
$\mathsf{Verify}(\pi_{\mathsf{amt}})$	253,384	0.00045609	1.41

TABLE IV
TPS FOR DIFFERENT VERIFICATION FUNCTIONS

Function	TPS
$\mathsf{Verify}(\pi_{\mathsf{buyer}})$	69.54
$\mathsf{Verify}(\pi_{\mathsf{supplier}})$	73.42
$\mathsf{Verify}(\pi_{\mathsf{amt}})$	80.06
$\mathsf{Verify}(\pi_{\mathsf{product}})$	75.30

2) TPS: TPS is a measure of the number of transactions that can be processed by the system in one second. It is an important metric for evaluating the performance and scalability of ZKVeil. We measure TPS for different verification functions, including identity verification, product verification, and amount verification. The TPS is calculated based on the average time taken to process each verification function.

The TPS for different verification functions is shown in Table IV. The TPS values indicate the number of transactions that can be processed per second for each verification function. According to the results, the TPS of all verification functions remains within a reasonable range. In particular, the performance of amount verification and identity verification is prominent, with the highest and higher TPS, respectively.

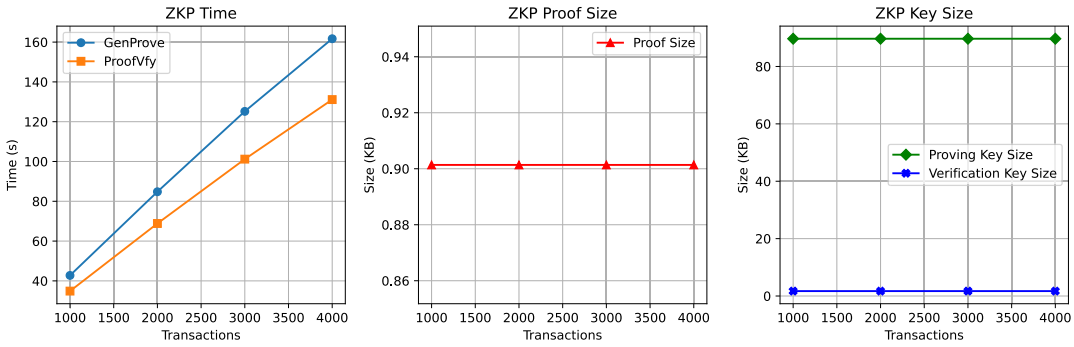
3) Time and Memory Cost of ZKVeil: ZK-SNARKs are employed in ZKVeil to enable zero-knowledge verification of regulatory compliance. We analyze the time and memory costs of generating and verifying ZKPs for identity, qualification, and transaction amount. The time cost is measured in seconds, while the memory cost is reported in kilobytes (KB).

The time and memory costs are shown in Figure 3. Notably, the generation of proving and verification keys needs to be executed only once during the initialization and is independent of subsequent transaction executions. This one-time setup mitigates recurring computational costs during runtime.

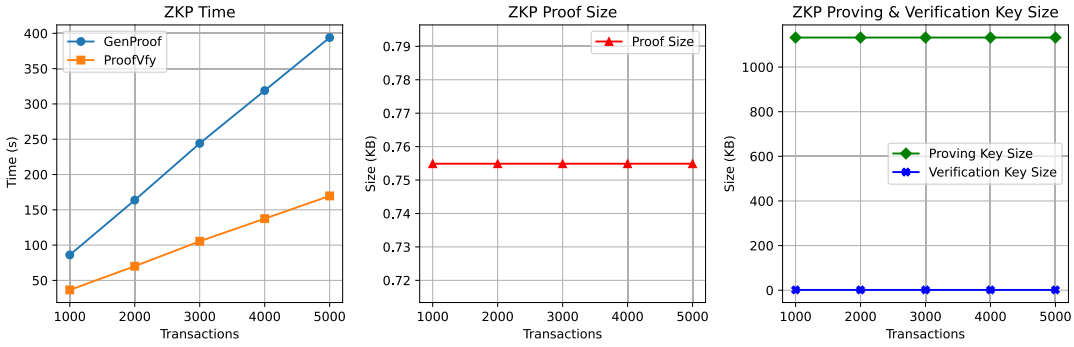
Experimental results reveal that the time required for proof generation and verification grows approximately linearly with the number of transactions. It can be reduced by parallel computing proof and batch verification of ZKP. The verification proof takes a short time, which is convenient for some regulatory nodes on the blockchain to verify. This low-latency verification is particularly suitable for on-chain enforcement by regulatory nodes, enabling them to perform efficient compliance checks without incurring a significant

[5]https://hardhat.org/
[6]https://zokrates.github.io/
[7]https://github.com/uport-project/ethr-did
[8]https://www.ccs.org.cn/ccswz/specialDetail?id=202406120326276668
[9]ETH/USD spot price retrieved from CoinMarketCap on December 13, 2025, available at https://coinmarketcap.com/currencies/ethereum/

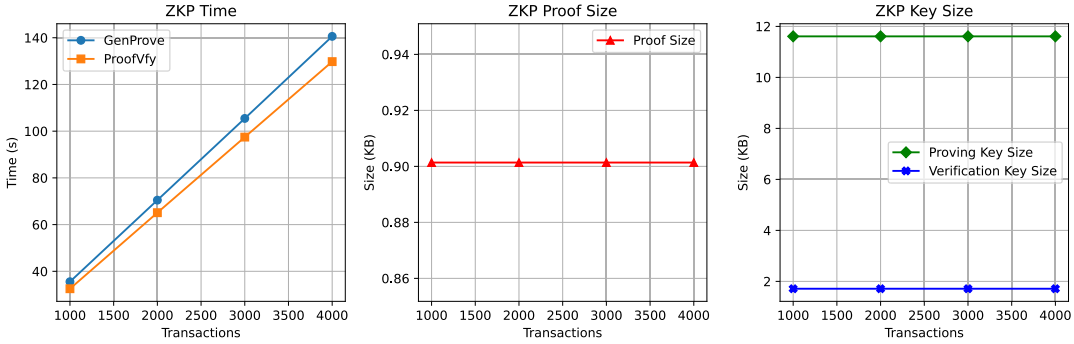
Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.



(a) Time and memory cost of π_{buyer}



(b) Time and memory cost of π_{product}



(c) Time and memory cost of π_{amt}

Fig. 3. Time and memory costs of various proofs.

computational burden. Moreover, the storage size of each proof remains compact, resulting in a low on-chain storage overhead. This aligns well with the storage constraints of public and private blockchains, ensuring that privacy-preserving compliance mechanisms do not compromise system efficiency.

Among various proofs, π_{product} incurs the highest time and memory costs. This is attributable to the complexity of the constraints involved in validating multidimensional specification parameters, which require a greater number of arithmetic circuit gates and constraint checks.

4) End-to-End Performance Comparison Results: To quantify the performance overhead introduced by the privacy-preserving compliance mechanisms in ZKVeil, we implement two baselines under the same blockchain configuration as ZKVeil. All baselines preserve an identical business workflow, ensuring that the observed performance differences are attributable to the privacy-preserving compliance mechanisms.

a) Plain-SC: The baseline removes all privacy-preserving mechanisms. All compliance-relevant attributes (identity, product specifications, and transaction amounts) are disclosed to the smart contract in plaintext, and the contract directly evaluates the regulatory predicates. This baseline captures the best-case performance when privacy is not required.

b) Encrypt-then-Regulate (ETR): This baseline is implemented following the general idea in [12], [13], and [14], which protects sensitive information by applying cryptographic techniques to encrypt data before it is recorded on the blockchain. This baseline encrypts sensitive qualifications, product specifications, transaction amounts using public-key encryption, and records ciphertexts on-chain. A trusted regulatory authority decrypts the ciphertexts off-chain, performs compliance checks on plaintext data, and submits a signed attestation indicating pass or fail. The smart contract verifies the regulator's signature and updates the transaction state accordingly.

c) Metrics: We evaluate end-to-end performance using two metrics: total gas, representing the aggregate gas cost

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

TABLE V
END-TO-END PERFORMANCE COMPARISON

Scheme	Total Gas	End-to-End Latency (s)
Plain-SC	744,216	3.42
ETR	1,224,246	5.26
ZKVeil	1,511,205	6.35

incurred by all protocol transactions, and end-to-end Latency, which captures the time duration from the initiation of a supply chain transaction to the completion of fund settlement.

Table V presents an end-to-end performance comparison among Plain-SC, ETR, and ZKVeil, in terms of total gas consumption and overall latency. As expected, Plain-SC achieves the lowest cost since it executes the business logic directly on-chain without introducing any cryptographic mechanisms. This result serves as a baseline representing the minimal overhead of finishing a supply chain transaction.

Compared to Plain-SC, ETR incurs a moderate increase in overhead, which corresponds to additional gas consumption and an increase in latency, which can be attributed to the introduction of encrypted data handling and additional regulatory attestation logic. Notably, the latency of ETR is measured under an optimistic setting, where encrypted data are uploaded on-chain and immediately decrypted and inspected by the regulator off-chain. In practice, such off-chain inspection may involve additional delays depending on regulatory workflows and off-chain processing. Moreover, it fundamentally relies on trusted off-chain decryption and plaintext inspection, introducing a centralized compliance intermediary and weakening privacy and auditability guarantees.

ZKVeil exhibits the highest cost among the three schemes. Compared to ETR, the additional overhead is 23.4% in gas and 20.7% in latency. This overhead mainly stems from the on-chain verification of ZKPs, as well as the additional cryptographic commitments. By shifting compliance verification from off-chain plaintext inspection to on-chain ZKP verification, ZKVeil trades additional gas consumption and on-chain latency for stronger decentralization, auditability, and privacy guarantees, while eliminating reliance on trusted intermediaries. Despite the increased cost, the performance results demonstrate that ZKVeil remains practically efficient, which is acceptable for typical blockchain-based supply-chain procurement scenarios that are not latency-critical. Overall, the end-to-end evaluation highlights a trade-off between privacy and performance.

VIII. DISCUSSION

In this section, we deeply discuss the security goals proposed in Section IV-C.

A. Identity Privacy

Each participant's identity is endorsed through VCs issued by trusted authorities. Participants generate ZKPs of set membership or attribute bounds without revealing the actual attribute values. By the zero-knowledge property, no information about the hidden credentials (e.g., age, license type) is leaked beyond what is explicitly proven. This ensures identity privacy across transactions.

B. Product Specification Privacy

The supplier generates ZKPs to demonstrate compliance with specific regulatory policies without revealing sensitive product specifications. According to the product manufacturing specification, the circuit is designed, and then the proof key and verification contract are generated. The supplier ensures that only proof is uploaded to the blockchain, while the actual product specification data remains off-chain and confidential.

C. Transaction Amount Privacy

The transaction amount is hidden using a cryptographic commitment scheme that satisfies binding and hiding properties. The binding property ensures that the committed value cannot be changed once committed, thereby preserving integrity, while the hiding property guarantees that the committed value remains secret to any party lacking the opening information. These properties rely on standard cryptographic hardness assumptions, such as the computational difficulty of the discrete logarithm or hash-collision resistance.

D. Data Authenticity

The authenticity of identity and product information is endorsed through digital signatures of trusted authentication authorities. The digital signatures are generated using asymmetric cryptography, where the private key is securely held by the authority. The corresponding public key is widely distributed and can be used to verify the authenticity of the signatures. This ensures that only authorized parties can issue verifiable credentials, and any tampering with the data can be detected through signature verification.

E. Regulatory Compliance

All proofs enforce compliance verification on-chain without revealing sensitive data. The smart contract verifies the proofs and ensures that the transaction adheres to regulatory policies. If any proof fails verification, the transaction is rejected. This mechanism ensures that only compliant transactions are executed on-chain, while non-compliant ones are automatically invalidated.

IX. CONCLUSION

In this paper, we introduced ZKVeil, a privacy-preserving compliance verification scheme for supply chain transactions. ZKVeil ensures the privacy and security of sensitive information, such as buyer and supplier qualifications, transaction amounts, and product specifications, while maintaining regulatory compliance. In the future, in addition to considering supply chain transactions between suppliers and buyers, we will explore more scenarios in the supply chain, such as considering trade compliance in cross-border transportation, and regulating the compliance of supply chain financial products.

REFERENCES

[1] J. Pennekamp et al., "An interdisciplinary survey on information flows in supply chains," *ACM Comput. Surveys*, vol. 56, no. 2, pp. 1–38, Feb. 2024.

[2] H. Zhang, Y. Lv, S. Zhang, and Y. D. Liu, "Digital supply chain management: A review and bibliometric analysis," *J. Global Inf. Manag. (JGIM)*, vol. 32, no. 1, pp. 1–20, 2024.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.

[3] J. J. Hathaliya and S. Tanwar, "A systematic survey on security and privacy issues of medicine supply chain: Taxonomy, framework, and research challenges," *Secur. PRIVACY*, vol. 7, no. 4, pp. 1–36, Jul. 2024, Art. no. 377.

[4] Y. Ding, Z. Pei, and C. Cao, "When two chains meet: Optimizing the design of blockchain-enabled supply chain networks," *Expert Syst. Appl.*, vol. 261, Feb. 2025, Art. no. 125481.

[5] M. Zhang, N. Wang, H. Liu, and Z. Zhang, "Cap allocation rules for an online platform supply chain under cap-and-trade regulation," *Int. Trans. Oper. Res.*, vol. 31, no. 4, pp. 2559–2590, Jul. 2024.

[6] Q. Zhu, Y. Sun, S. K. Mangla, S. Arisian, and M. Song, "On the value of smart contract and blockchain in designing fresh product supply chains," *IEEE Trans. Eng. Manag.*, vol. 71, pp. 10557–10570, 2024.

[7] S. Sahoo, A. Kumar, R. Mishra, and P. Tripathi, "Strengthening supply chain visibility with blockchain: A PRISMA-based review," *IEEE Trans. Eng. Manag.*, vol. 71, pp. 1787–1803, 2024.

[8] M. Asante, G. Epiphaniou, C. Maple, H. Al-Khateeb, M. Bottarelli, and K. Z. Ghafoor, "Distributed ledger technologies in supply chain security management: A comprehensive survey," *IEEE Trans. Eng. Manag.*, vol. 70, no. 2, pp. 713–739, Feb. 2023.

[9] W. Liang, Y. Liu, C. Yang, S. Xie, K. Li, and W. Susilo, "On identity, transaction, and smart contract privacy on permissioned and permissionless blockchain: A comprehensive survey," *ACM Comput. Surveys*, vol. 56, no. 12, pp. 1–35, Dec. 2024.

[10] L. Bader et al., "Blockchain-based privacy preservation for supply chains supporting lightweight multi-hop information accountability," *Inf. Process. Manage.*, vol. 58, no. 3, May 2021, Art. no. 102529.

[11] C. Zhang, L. Zhu, C. Xu, K. Sharif, R. Lu, and Y. Chen, "APPB: Anti-counterfeiting and privacy-preserving blockchain-based vehicle supply chains," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13152–13164, Dec. 2022.

[12] B. B. Sezer, S. Topal, and U. Nuriyev, "TPPSUPPLY: A traceable and privacy-preserving blockchain system architecture for the supply chain," *J. Inf. Secur. Appl.*, vol. 66, May 2022, Art. no. 103116.

[13] C. Xu, Y. Qu, Y. Xiang, T. H. Luan, and L. Gao, "An optimized privacy-protected blockchain system for supply chain on Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 9019–9030, Mar. 2024.

[14] J. Li, Z. Wang, S. Guan, and Y. Cao, "ProChain: A privacy-preserving blockchain-based supply chain traceability system model," *Comput. Ind. Eng.*, vol. 187, Jan. 2024, Art. no. 109831.

[15] G. Yu et al., "Enabling attribute revocation for fine-grained access control in blockchain-IoT systems," *IEEE Trans. Eng. Manag.*, vol. 67, no. 4, pp. 1213–1230, Nov. 2020.

[16] K. Fan et al., "A secure and verifiable data sharing scheme based on blockchain in vehicular social networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 5826–5835, Jun. 2020.

[17] N. C. K. Yiu, "Toward blockchain-enabled supply chain anti-counterfeiting and traceability," *Future Internet*, vol. 13, no. 4, p. 86, Mar. 2021.

[18] M. Lou, X. Dong, Z. Cao, and J. Shen, "SESCF: A secure and efficient supply chain framework via blockchain-based smart contracts," *Secur. Commun. Netw.*, vol. 2021, pp. 1–18, Jan. 2021.

[19] S. K. Dwivedi, R. Amin, and S. Vollala, "Blockchain based secured information sharing protocol in supply chain management system with key distribution mechanism," *J. Inf. Secur. Appl.*, vol. 54, Oct. 2020, Art. no. 102554.

[20] S. Jangirala, A. K. Das, and A. V. Vasilakos, "Designing secure lightweight blockchain-enabled RFID-based authentication protocol for supply chains in 5G mobile edge computing environment," *IEEE Trans. Ind. Informat.*, vol. 16, no. 11, pp. 7081–7093, Nov. 2020.

[21] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Proc. Annu. Int. Cryptol. Conf. (Crypto)*, 2004, pp. 41–55.

[22] Mohit, S. Kaur, and M. Singh, "Design and implementation of blockchain-based supply chain framework with improved traceability, privacy, and ownership," *Cluster Comput.*, vol. 27, no. 3, pp. 2345–2363, Jun. 2024.

[23] Z. Zhou, J. Gong, Y. He, and Y. Zhang, "Software defined machine-to-machine communication for smart energy management," *IEEE Commun. Mag.*, vol. 55, no. 10, pp. 52–60, Oct. 2017.

[24] S. A. Abeyratne and R. P. Monfared, "Blockchain ready manufacturing supply chain using distributed ledger," *Int. J. Res. Eng. Technol.*, vol. 5, no. 9, pp. 1–10, Sep. 2016.

[25] J. Li, D. Han, T.-H. Weng, H. Wu, K.-C. Li, and A. Castiglione, "A secure data storage and sharing scheme for port supply chain based on blockchain and dynamic searchable encryption," *Comput. Standards Interfaces*, vol. 91, Jan. 2025, Art. no. 103887.

[26] B. Yong, J. Shen, X. Liu, F. Li, H. Chen, and Q. Zhou, "An intelligent blockchain-based system for safe vaccine supply and supervision," *Int. J. Inf. Manage.*, vol. 52, Jun. 2020, Art. no. 102024.

[27] B. Wen, Y. Wang, Y. Ding, H. Liang, C. Yang, and D. Luo, "Towards privacy-preserving and practical vaccine supply chain based on blockchain," in *Proc. 18th Asia Joint Conf. Inf. Secur. (AsiaJCIS)*, Aug. 2023, pp. 39–46.

[28] H. Huang, W. Kong, S. Zhou, Z. Zheng, and S. Guo, "A survey of state-of-the-art on blockchains: Theories, modelings, and tools," *ACM*, pp. 44:1–44:42, 2020.

[29] C. Mazzocca, A. Acar, S. Uluagac, R. Montanari, P. Bellavista, and M. Conti, "A survey on decentralized identifiers and verifiable credentials," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 6, pp. 3641–3671, Dec. 2025.

[30] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in *Proc. Work Shafi Goldwasser Silvio Micali*, 1985, pp. 291–304.

[31] L. Zhou, A. Diro, A. Saini, S. Kaisar, and P. C. Hiep, "Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities," *J. Inf. Secur. Appl.*, vol. 80, Feb. 2024, Art. no. 103678.

[32] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying program executions succinctly and in zero knowledge," in *Proc. 33rd Annu. Cryptol. Conf.*, Aug. 2013, pp. 90–108.

[33] Z. Guan, Z. Wan, Y. Yang, Y. Zhou, and B. Huang, "BlockMaze: An efficient privacy-preserving account-model blockchain based on zk-SNARKs," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1446–1463, May 2022.

Authorized licensed use limited to: Hanyang University. Downloaded on July 21,2026 at 06:40:37 UTC from IEEE Xplore. Restrictions apply.